EC551: Advanced Digital Design with Verilog and FPGAs
Fall 2026
Prof. Douglas Densmore
Programming Assignment 1 – Logic Synthesis Engine
Rev 1.0 | **Due:** Friday, 10/23 by 11:59 PM

Overview
This programming assignment requires you to design and implement a custom **Logic Synthesis Engine**. Your program will accept a Boolean algebraic function or digital combinational circuit, optimize/manipulate it across various objective functions, report logic metrics, and automatically generate a structural Verilog file and/or netlist. Finally, you will synthesize and implement your generated Verilog on the FPGA board.

- **Team Structure:** Groups of **2 students**.
- **Programming Language:** Any language/environment of your choice (e.g., Python, C++, Rust). See AI policy.
- **Demonstration:** Groups will sign up for a demo slot with Prof. Densmore to demonstrate the software and live FPGA hardware functionality.

1. Input Requirements
Your tool must accept input via **at least one** of the following interfaces (you may support multiple):

- **a.** A file (e.g., via CLI argument or standard file picker)
- **b.** Direct command-line interaction
- **c.** A Graphical User Interface (GUI)

Supported Input Representations
Your program must handle inputs representing either:

1. **A Boolean Algebraic Function**
    - *Required:* Sum-of-Products (SOP) representation.
    - *Optional:* Product-of-Sums (POS), multi-level logic expressions, etc.
2. **A Digital Combinational Logic Circuit**
    - Graph-based circuit representation (nodes as logic gates/operators, edges as interconnects). You can use a format of your choosing or design (e.g. XML, JSON, etc.).

Note: A single input format does not need to support both algebraic and graph representations, but your engine must accept input files or formats capable of representing either type.

Suggested Formats:

You are strongly encouraged to leverage or adapt existing EDA standards:

- **AIG** (And-Inverter Graph)
- **BLIF** (Berkeley Logic Interchange Format)
- **EQN** (Equation Format)
- **PLA** (Programmable Logic Array / Espresso format)

Useful Resources:

- UC Berkeley ABC Tool
- UC Berkeley Espresso Tool

Feel free to explore other formats! Post interesting findings or parsers to Piazza.

2. Logic Synthesis Engine Features

Once an input is loaded, your tool must execute commands to perform the following **12 operations**:

1. **Canonical SOP:** Compute and display the canonical Sum-of-Products form.
2. **Canonical POS:** Compute and display the canonical Product-of-Sums form.
3. **Inverse Canonical SOP:** Compute and display the canonical SOP form for the inverse function ($\bar{F}$).
4. **Inverse Canonical POS:** Compute and display the canonical POS form for the inverse function ($\bar{F}$).
5. **Literal Minimized SOP:** Return a minimized SOP representation (e.g., using Quine-McCluskey, Espresso-like algorithms, or K-map minimization).
    - *Metric:* Report the number of literals saved vs. the canonical SOP version.
6. **Literal Minimized POS:** Return a minimized POS representation.
    - *Metric:* Report the number of literals saved vs. the canonical POS version.
7. **Prime Implicants:** Report the total count of Prime Implicants.
8. **Essential Prime Implicants:** Report the total count of Essential Prime Implicants.
9. **ON-Set Minterms:** Report the total number of ON-Set minterms (Σm).

10. **OFF-Set Maxterms:** Report the total number of OFF-Set maxterms (ΠM).

11. **Custom Command #1:** A feature/command of your choice *(e.g., hazard detection, delay estimation, XOR/XNOR factorization, truth table exporter)*.

12. **Custom Command #2:** A second feature/command of your choice.

3. Output Requirements
Your tool must provide the following outputs:

1. **Display/Export:** Output text results to a file, stdout, or GUI pane.

2. **Formatting Results:**

    - **Boolean Expressions:** Output clear expressions for operations 1–6.

    - **Quantitative Metrics:** Clear numeric output for operations 7–10.

    - **Custom Outputs:** Meaningful output corresponding to your choices for operations 11–12.

3. **Structural Verilog Generation:** Automatically generate a valid structural Verilog .v file by instantiating primitive gates (and, or, not, nand, nor, etc.) and wiring them to implement the minimized circuit.

4. **FPGA Deployment:**

    - Synthesize and map the structural Verilog generated onto your FPGA board.

    - Be creative with input/output mapping! You may map inputs to physical FPGA switches/buttons and outputs to LEDs/7-segment displays. **Note: it is ok if for large input spaces not all inputs can be manually controlled.**

    - Hand-written Verilog wrappers are permitted to connect your generated structural module to physical IO pins.

4. Submission & Evaluation

Repository Requirements

Submit a link to a **public GitHub repository** to Prof. Densmore containing:

1. **Source Code:** Well-structured and commented codebase.
2. README.md **File:**
    - High-level project description and architecture overview.
    - Repository folder structure layout.
    - Step-by-step build, run, and usage instructions.

Live Demonstration

- Prof. Densmore will post a sign-up sheet for group demo slots.

- During your demo, you will:

    1. Show full engine functionality via command execution on custom test cases.

    2. Walk through code structure and explain key implementation decisions.

    3. Show the automatically generated Verilog synthesized and running live on the FPGA hardware.

5. AI Policy (per Syllabus)

Generative AI tools **can** be used for this programming assignment, provided you **cite your usage** (in your repository README.md or code headers) and are fully prepared to explain and defend any generated logic or architecture during your demo.